

Technical Report for the IMU Odometry Challenge

Trajectory-Context Inertial Velocity Estimation with Metric-Shaped Training

Kaggle team name: **Hack2Publish**

Md. Hamid Hosen¹, Esfer Sami², Kahakashan Ashraf², and Foysal Emon Shanto²

Abstract—This report describes the Hack2Publish entry to the IMU Odometry Challenge. We train one network, with a single set of weights, that takes raw 6-axis IMU at 200 Hz and outputs the mean body-frame velocity of every 1 s window for all four platforms. The main idea is to let the network see much more than one window: at inference we cut each test trajectory into 16 s chunks, predict a dense 20 Hz velocity for the whole chunk, and average the overlapping chunk predictions. The network is a strided convolutional stem, eight dilated temporal-convolution blocks and a small three-layer transformer (3.9M parameters); a set of hand-computed strap-down features is added to the stem output. Training samples platforms and trajectories the way the metric averages them, and adds a loss on the integrated velocity error so that the ATE_{20} term is optimised as well as the AVE term. Augmentation is kept physically exact: random re-mounting of the sensor (applied to the IMU and to the velocity label), time dilation using the gravity vector available at training time, and bias/scale/noise. Only the challenge data was used. We chose every hyper-parameter on the released val split and then refit the final model on train+val for 160 epochs (EMA plus weight averaging over the last 20 epochs). On the official scoring service the submitted model reaches a TartanIMU Score of 0.219 over the 89 test sequences (macro AVE 0.180 m/s, ATE_{20} 0.563 m). On the final (private) leaderboard it scores 0.16141, seventh of 132 teams (0.28609 and tenth on the public leaderboard). We also document two properties of the drone data that we believe limit every IMU-only entry, and a measured leaderboard noise of ± 0.01 – 0.02 between identical recipes.

I. INTRODUCTION

Three things make a general inertial odometer difficult. First, the physics is only partly observable: a 6-axis IMU measures specific force and angular rate, so gravity, sensor biases and the mounting of the sensor are mixed into the same six numbers as the motion we want, and slow motion (a hovering drone, a person standing still) hardly shows up at all. Second, the four embodiments in this challenge differ by an order of magnitude in speed, vibration and dynamics, and the test set does not say which is which, so the network has to work that out from the signal. Third, the ground truth was produced by different capture systems, and, as we found during the challenge, they are not equally consistent with the IMU streams.

Our approach was shaped by one measurement we made early on: the per-window velocity is strongly correlated between neighbouring windows (lag-1 correlation 0.85 for the

car, 0.78 for the dog, 0.56 for the human), and an oracle that simply averages the true velocities of the two neighbouring windows already halves the all-zero error. A single second of IMU cannot estimate gyroscope bias, low-frequency tilt or the embodiment with any confidence; sixteen seconds can. Since the test trajectories are provided whole, we built the entire solution around trajectory context. The other two ingredients are a training procedure that mirrors how the score is averaged, and augmentation that respects the physics of the sensor. What we think distinguishes the entry is less any single component than the discipline of selecting everything on the held-out split and then explaining, with measurements, where the remaining error comes from. Our contributions are (i) the context model and training recipe, (ii) an experimental record of what helped and what did not, and (iii) two data findings about the drone split (Sec. IV-D) that we hope are useful to the organisers.

II. METHOD

A. Overall Architecture

The pipeline is drawn in Fig. 1, and Fig. 2 follows one chunk from raw IMU to the challenge score. The input is a chunk of $T=16$ consecutive windows, i.e. 3200×6 raw samples $[a_x, a_y, a_z, \omega_x, \omega_y, \omega_z]$ with gravity retained. We do not subtract a mean; each channel is only divided by a fixed constant (3 m/s^2 for the accelerometer, 0.4 rad/s for the gyroscope). Keeping the mean matters twice: gravity is the only tilt cue the network has, and a zero-mean rotation augmentation stays exact.

A two-layer strided convolution (kernel 9, stride 2; kernel 11, stride 5) turns the 200 Hz signal into 20 Hz tokens of width 192. In parallel we compute, without learned parameters, a small set of strap-down features on the same chunk: the gyroscope is integrated into a relative orientation for every token (a parallel prefix product of per-token rotation matrices), the accelerometer is rotated into the frame of the chunk start, its chunk mean is used as a gravity estimate, and the de-gravitated acceleration and its running integral are rotated back into the current body frame. Because one of the drone sources has an inverted gyroscope z axis (Sec. IV-D), we compute these features for both signs of ω_z and let the network decide; a 1×1 projection adds the 18 channels to the tokens. Eight residual blocks follow, each a depthwise convolution (kernel 5, dilations 1, 2, 4, 8, 16, 32, 1, 2) and a pointwise MLP with GroupNorm, giving a receptive field of roughly 13 s. A three-layer transformer encoder (4 heads,

¹Md. Hamid Hosen, team leader. mdhamidhosen4@gmail.com

²Esfer Sami, Kahakashan Ashraf and Foysal Emon Shanto, team members.

learned positional embedding, pre-norm) then mixes the 320 tokens of the chunk, and a linear head outputs a velocity per token, i.e. a dense 20 Hz velocity. The per-window prediction is the mean of the 20 tokens inside the window. An attention-pooled descriptor also feeds a 4-way platform classifier, but this head only exists in the training graph; it is never evaluated at inference and nothing downstream depends on it. There is no platform input, no routing and no per-platform component anywhere in the network.

The choices made with cross-platform generalisation in mind were: no mean normalisation (see above), bidirectional context (bias and tilt are effectively estimated over 16 s rather than 1 s), and sampling that gives every platform the same number of gradient steps regardless of how many hours of data it has.

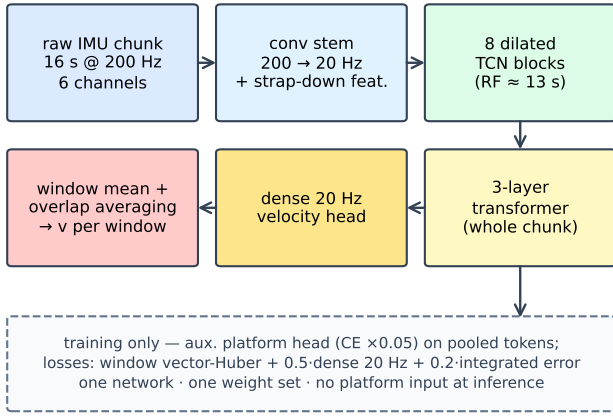


Fig. 1. The pipeline. One network and one weight set for all platforms; the platform head in the dashed box is used only as a training signal.

B. Data Processing

Windows are used exactly as the organisers define them (window k is `imu[200k:200k+200]`). Whole trajectories are kept in memory and a training sample is a random 16-window chunk of one trajectory; trajectories shorter than 16 windows are padded by repeating the last sample and the padded windows are masked out of the loss. The 20 Hz targets are 10-sample means of `vel.body`. No filtering, re-sampling, bias correction or calibration is applied, at training or at test time.

Augmentation is applied on the GPU, per chunk. (a) *Re-mounting the sensor*: a random rotation (uniform axis, angle uniform in $[0, 15^\circ]$, or $[0, 45^\circ]$ for drone chunks) is applied to the accelerometer, the gyroscope and the velocity label alike, which is exactly what a differently mounted sensor would have measured. (b) *Time dilation*: the chunk is resampled by a factor $s \sim \log U(1/1.3, 1.3)$; the velocity label and the gyroscope are scaled by s , and the dynamic part of the acceleration by s^2 , while the gravity part, which we know from the training quaternion, is kept. This produces realistic faster and slower versions of every flight. (c) Sensor errors: accelerometer scale $1 + \mathcal{N}(0, 0.02)$ and bias

$\mathcal{N}(0, 0.15 \text{ m/s}^2)$, gyroscope scale $1 + \mathcal{N}(0, 0.02)$ and bias $\mathcal{N}(0, 0.02 \text{ rad/s})$, white noise of 0.05 m/s^2 and 0.004 rad/s .

C. Training Data

Table I lists everything that was used. There is nothing beyond the challenge data: no external datasets, no synthetic data, no pretrained weights (including the released baseline) and no pseudo-labels. During model development the released `train` split (395 trajectories) was used for training and the released `val` split (80 trajectories) only for scoring. The final model was refit on the union with the schedule that had been fixed on `val`. Test files were opened only by the inference script.

TABLE I
DATASETS USED FOR MODEL DEVELOPMENT AND TRAINING.

Dataset	Type	Usage
Challenge train (395 traj.)	Official	Training in all selection runs; training of the final model
Challenge val (80 traj.)	Official	Scoring of selection runs (never trained on there); added to training for the final refit

D. Training and Optimization

The loss is $L = L_{\text{win}} + 0.5 L_{\text{dense}} + 0.2 L_{\text{drift}} + 0.05 L_{\text{plat}}$. L_{win} is a Huber loss ($\beta = 0.25 \text{ m/s}$) on the Euclidean norm of the per-window velocity error, which is the AVE up to the Huber knee; L_{dense} is the same loss on the 20 Hz output; $L_{\text{drift}} = \frac{1}{T} \sum_t \|\sum_{k \leq t} (\hat{v}_k - v_k)\| / \sqrt{t}$ penalises the accumulated body-frame error inside the chunk, which is the kind of correlated bias that ATE_{20} measures and a per-window loss does not see; L_{plat} is the cross-entropy of the auxiliary platform head. The weights were set by hand at the start and never tuned.

We use AdamW ($\beta_1=0.9$, $\beta_2=0.99$, weight decay 0.02), a OneCycle schedule with peak learning rate 1.5×10^{-3} and 8 % warm-up, batches of 64 chunks, 250 optimiser steps per epoch and gradient clipping at 2.0. An exponential moving average of the weights (decay 0.998) is kept during training, and the final weights are the plain average of the EMA weights taken at the end of each of the last 20 epochs. Sampling: a platform is drawn uniformly, then a trajectory uniformly within that platform, then a random chunk position. Selection runs lasted 60 epochs; the final refit 160 epochs. Training is a single stage; there is no curriculum, no fine-tuning, no domain adaptation and no platform-specific specialisation. The final checkpoint is simply the averaged weights at the end of the fixed schedule – with `val` inside the training set there is nothing to select on, and we did not want to select on the leaderboard (Sec. VI-B). Selection runs were trained on an Apple M5 laptop GPU (about 2–3 min per epoch); the ranked checkpoint was trained by a Kaggle script kernel on a Tesla T4 in 3.6 h.

E. Inference

For each test trajectory we read only the `imu` array and the window order. Chunks of 16 windows are slid over

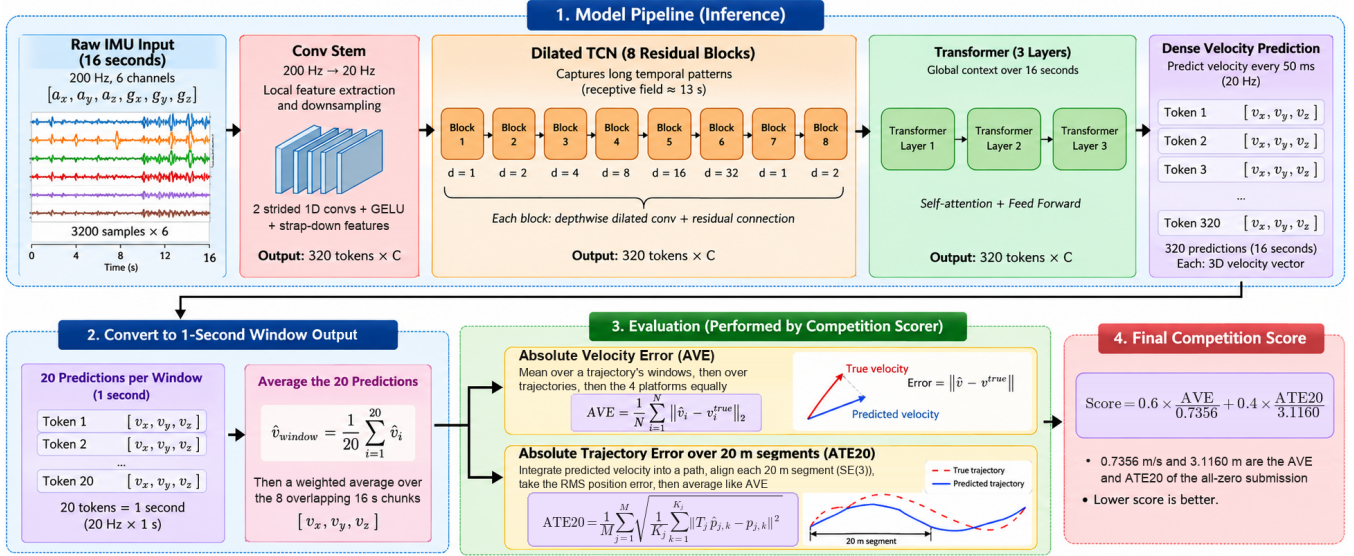


Fig. 2. From raw IMU to the challenge score. (1) The network reads a 16 s chunk and predicts a velocity for each of its 320 tokens (20 Hz). (2) The 20 token predictions inside a 1 s window are averaged, and each window’s estimate is then a Hann-weighted average over the 8 overlapping chunks (stride 2 windows) that cover it. (3) The organisers’ scorer computes AVE and ATE₂₀ (each 20 m segment SE(3)-aligned, RMS position error), averaged per trajectory, then per platform, then over the four platforms. (4) Each term is divided by the all-zero submission’s value, so all-zero scores 1.000.

the trajectory with a stride of 2 windows; every chunk is passed through the network once; the per-window outputs of overlapping chunks are averaged with a raised-cosine weight (plus a small floor) so that a window is trusted most from the chunk in which it lies near the centre. Trajectories shorter than 16 windows are padded by repeating the last sample and the padding is discarded afterwards. Trajectories are never concatenated. There is no test-time augmentation, no ensembling, no scaling, clipping, smoothing or calibration, and no test-time adaptation. The method is offline: it uses past and future samples of the same trajectory.

F. Model Complexity and Runtime

Table II gives the numbers. The whole 89-trajectory test set takes 26 s on a Tesla T4; on a 10-core CPU the same run takes 30 min because of the 8-fold chunk overlap. The method as submitted is not causal; a causal variant would need a different context design and we did not build one.

TABLE II

MODEL COMPLEXITY AND RUNTIME STATISTICS (RANKED ENTRY).

Property	Value
Number of parameters	3.9 M (second entry: 1.8 M)
Training hardware	Kaggle Tesla T4 (ranked run); Apple M5 laptop GPU (all selection runs)
Training time	3.6 h for 160 epochs $\times$ 250 steps $\times$ 64 chunks
Inference hardware	one GPU with <2 GB of memory, or CPU
Inference speed (Hz)	$\approx 1,200$ windows/s on a Tesla T4 (all 30,644 test windows in 26 s including loading); ≈ 17 windows/s on a 10-core laptop CPU (30 min)
Peak memory	<2 GB GPU; 0.6 GB RAM measured on CPU (whole test set)
Online / real time	no (offline, bidirectional context)

III. DEVELOPMENT EXPERIMENTS

Unless stated otherwise, the numbers in this section are TartanIMU scores on the held-out `val` split (80 trajectories, organisers’ scorer) of models trained on `train` only. Fig. 3 shows the training curves of the main runs.

Architectures. We started from the per-window 1-D ResNet of the public starter kernel, which reaches about 0.30 on val. Our first context model (width 128, 2-layer transformer, 16 s chunks, 30 epochs) reached 0.2247; that single change is by far the largest gain we saw. A wider trunk with 32 s chunks was worse at the same number of epochs (0.2293), so 16 s stayed. A bidirectional GRU in place of the transformer gave the same accuracy but was three times slower on our hardware. Widening the trunk to 192 channels and three transformer layers, at 60 epochs, improved val from 0.2030 to 0.1987; the improvement was spread evenly over the platforms.

Schedule length. The same recipe trained for 60 instead of 30 epochs went from 0.2233 to 0.2030 (drone-B AVE 0.328 $\rightarrow$ 0.271 m/s, car 0.129 $\rightarrow$ 0.117, human 0.085 $\rightarrow$ 0.073). On the public leaderboard the train+val refits went 0.3625 (30 epochs), 0.3466 (60), 0.2975 (100), 0.2870 (160); a 240-epoch run of the wide model did not improve further (0.2897).

Sampling. Drawing trajectories uniformly within a platform instead of in proportion to their length took val from 0.2030 to 0.1982. The metric averages per trajectory, so a 13-window drone clip counts as much as a 35-minute car drive; length-proportional sampling had been under-training exactly the short clips that carry most of the error. Raising the drone share of the sampling to 40 % (0.2013) or adding a vibration-amplitude augmentation (0.2065) was worse than plain uniform sampling. The ranked entry combines uniform

sampling with the wider trunk; we did not have time to validate that combination as a separate 60-epoch run, so its only scores are those of the train+val refit itself (0.2861 public, 0.1614 private, 0.219 full test).

Drone-targeted augmentation. Rotations up to 45° for drone chunks, a fourfold oversampling of the racing-drone source and time dilation together moved val from 0.2247 to 0.2233, but the racing-drone trajectories themselves did not move at all (AVE 0.674 $\rightarrow$ 0.676 m/s). That was the first hint that their error is not a matter of training signal.

Things that did not work. The strap-down features on their own gave no measurable gain: integrating the raw gyroscope over 16 s drifts by 5–20 m/s because of uncompensated bias, so the network’s own short-horizon integration is already better (we kept the features because they cost nothing). A learned per-chunk time-shift head, supervised with IMU-to-ground-truth offsets we measured on the training set, predicted the offsets of unseen recordings well (for example -66 and -48 ms predicted against -40 and -70 ms measured) but did not improve the score. Rotation test-time augmentation (0.2246 against 0.2247), a 3-window moving average (ATE worse on every platform) and a denser overlap at inference (identical) changed nothing. Averaging the predictions of two to four runs improved val to 0.2171 and the public score to 0.354 from 0.363, but that is not a single weight set and none of our ranked entries uses it.

Overall, more epochs and better sampling helped every platform; nothing we tried helped one platform at the expense of another; and model scale helped a little on val but did not show on the leaderboard.

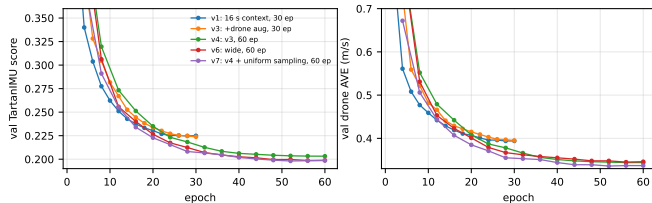


Fig. 3. Val score (left) and val drone AVE (right) during training for the main selection runs. All runs are trained on `train` only and scored on the held-out `val` split every 2–4 epochs.

IV. EXPERIMENTS

A. Official Challenge Performance

Tables III–V come from the official scoring service, run on the CSV of our ranked Kaggle submission 56402554, md5 = **fd12d4f25d7b52d44aaa4e4a14534378**. The service still lists our team under its earlier Kaggle name, “Md. Hamid Hosen” (we renamed the team to Hack2Publish on 21 September and the whitelist had not refreshed by the time of scoring); the md5 identifies the file regardless. The full-test score is 0.21903; on Kaggle it scores 0.16141 on the private leaderboard (final rank 7 of 132 teams) and 0.28609 on the public leaderboard (rank 10). Our second selected entry, submission 56332266 (md5 97a6e86dba7dfc19929e050bef5857b0), scores

0.22189 on the same service (private leaderboard 0.16625; AVE 0.181 m/s, ATE₂₀ 0.581 m). Table IV shows where the score comes from: AVE is 0.05–0.07 m/s on car, human and dog and 0.52 m/s on the drone, ATE₂₀ is 0.36–0.49 m against 1.04 m, and the drone term alone is 64 % of our score.

TABLE III

OVERALL CHALLENGE PERFORMANCE (OFFICIAL SCORING SERVICE, FULL 89-SEQUENCE TEST SET). THE TARTANIMU SCORE IS $0.6 \text{ AVE} / 0.7356 + 0.4 \text{ ATE}_{20} / 3.1160$, DIMENSIONLESS, *lower is better* (ALL-ZERO = 1.000). **RTE DOES NOT ENTER THE RANKING**; IT IS DEFINED IN THE CAPTION OF TABLE V. BASELINE NUMBERS ARE THE ORGANIZERS’ PUBLISHED FULL-TEST VALUES.

Method	Score ↓	ATE ₂₀ ↓	AVE ↓	RTE ↓
TartanIMU Baseline	0.538	1.261	0.461	–
Ours (Hack2Publish)	0.2190	0.563	0.180	0.868

TABLE IV

PERFORMANCE BY PLATFORM (OFFICIAL SCORING SERVICE). RTE IS DEFINED IN THE CAPTION OF TABLE V.

Platform	ATE ₂₀ ↓	AVE ↓	RTE ↓
Wheeled / Car	0.369	0.071	0.323
Handheld / Human	0.490	0.053	0.270
Legged / Quadruped	0.358	0.074	0.353
Aerial / Drone	1.037	0.522	2.528
Average	0.563	0.180	0.868

B. Per-Sequence Results

Table V in the appendix lists all 89 sequences. The pattern is a concentration rather than a spread. The car, human and dog sequences are uniform: their AVE sits within a narrow band whatever the length or speed of the sequence. Among the drone sequences, the ones that belong to the racing-drone source (short, 3–7 m/s, each flight recorded by two IMUs, see Sec. IV-D) have an AVE several times that of any other sequence, and a handful of them account for most of the drone term. At the other end, the slowest hovering drone sequences have an AVE about equal to the standard deviation of their own ground-truth velocity, which is what one gets by predicting the mean: hover drift of less than 0.2 m/s is simply not visible in the IMU.

C. Ablation Studies

Section III is our ablation record. Ordered by effect on val: trajectory context (−0.08), schedule length (−0.02), uniform trajectory sampling (−0.005), model scale (−0.004), drone augmentation (−0.001); strap-down features, the time-shift head, test-time augmentation and smoothing gave zero or negative changes.

D. Generalization Analysis

The held-out val split predicted the test set well: 0.198 on val against 0.219 on the full test set. The public split is

a different matter, 0.286, and the organisers’ baseline shows the same gap (0.538 full test, 0.637 public). Everything about this gap is in the drone data. Fig. 4 plots the per-trajectory val error of our recipe against the mean speed of the trajectory. Car, dog and human sit at 0.07–0.13 m/s regardless of speed. Drone-B, the second drone source, sits at 0.15–0.4 m/s, and its error is actually lower at 3–4 m/s than when hovering. Drone-A, the racing drone, sits at 0.45–1.0 m/s at every speed.

We spent a good part of the last week trying to understand drone-A and found two things, both measured on the released training and validation files.

Frame conventions. Drone-B’s gyroscope z axis is inverted with respect to the body frame of its quaternion and velocity: the ground-truth yaw rate equals $-\omega_z$ with $R^2 = 0.97$. Its accelerometer does not match the ground-truth specific force under any relabelling of the axes either (the R^2 at 2 Hz is negative; an unconstrained 3×3 fit gives axis gains between 0.6 and 2.1). The other three platforms and drone-A have identity conventions with R^2 between 0.97 and 0.99. The network copes with drone-B because the source is easy to recognise from the signal (its high-frequency accelerometer content is ten times lower), but whatever it learns about physics on drone-B’s 3.4 hours does not carry over to drone-A, which has about 30 minutes of data.

The racing-drone recordings. Every fast flight of drone-A appears twice, recorded by two different IMUs, one level and one pitched by about 45° (e.g. `drone_val_0005` and `0006`). Cross-correlating $|\omega|$ with the angular rate derived from the quaternions shows that the two IMU streams are offset from the ground truth by different amounts, about -10 ms for one IMU type and -40 to -70 ms for the other. After aligning the two recordings in time and fitting the best rotation between them, their velocity labels still disagree by 0.15–0.9 m/s, which corresponds to roughly 5° of inconsistency in the mounting at 4–7 m/s. No model that sees only the IMU can do better than that disagreement on this source, and about a quarter of the test drone trajectories are of this type. Several dog trajectories also have offsets of 40–150 ms, but at dog speeds these cost little.

We saw no negative transfer between platforms. Every improvement of the shared recipe improved every platform, and removing one platform from training never helped another.

E. Failure-Mode Analysis

The failures we can name are these. Racing-drone flights: errors of 0.5–1 m/s that look like a bias per recording, consistent with the timing and mounting inconsistencies above. Slow hover: a velocity oscillation of ± 0.15 m/s at 0.1 Hz produces about 0.1 m/s^2 of acceleration underneath $4\text{--}6 \text{ m/s}^2$ of vibration, and the network predicts the mean. Fast cars: the 25 val windows above 3 m/s have errors of 3–4 m/s, because nothing in the training data drives that fast. Two things remain open: a principled way to bring drone-B’s sensor convention in line with the rest of the data (it is not a simple relabelling of axes, so we could not canonicalise

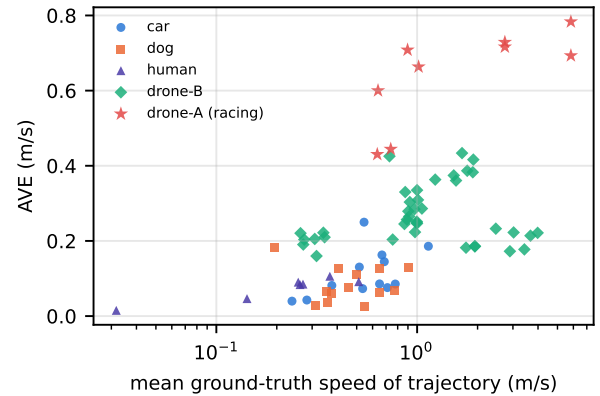


Fig. 4. Per-trajectory val AVE of the final recipe (train-only run) against the mean ground-truth speed of the trajectory. The racing-drone source is an outlier at every speed; the slowest drone-B sequences sit at the floor set by hover drift.

it), and any use of the fact that each racing flight is recorded twice.

V. INSIGHTS AND LESSONS LEARNED

Is a single universal inertial model feasible? For the part of the problem that is observable, yes, and without any specialisation. One 3.9M-parameter network with no platform input gets 0.05–0.07 m/s AVE on three platforms and around 0.27 m/s on ordinary drone flight, it identifies the embodiment on its own, and every change that helped one platform helped the rest. We never found a reason to add a platform-specific component.

Where is the bottleneck? Not in the model. The single largest remaining error comes from one drone source whose labels are inconsistent with each other at the level of tens of milliseconds and a few degrees. More capacity, more epochs and more augmentation left that error where it was while everything else kept improving. The second largest remaining error, hover drift, is a limit of the sensor, not of the learning.

What transfers? Reading whole trajectories, sampling the way the metric averages, and augmentation that respects the sensor physics do not depend on any particular IMU or vehicle and should carry over to other platforms and datasets. Anything the network has learned about a particular source’s axis convention or timing offset will not, and in a platform-blind test set with recognisable sources that kind of knowledge is rewarded. Per hour of data, the clean and slow drone-B recordings gave far more than the fast and noisy racing-drone recordings: the drone-B error kept falling with training, the racing-drone error did not move.

About the evaluation: with 45 drone sequences averaged per trajectory, a public split is dominated by a few short racing flights. We measured differences of ± 0.01 – 0.02 on the public leaderboard between runs of the same recipe on different GPUs, which is larger than most of the differences we were trying to detect; val, with all its windows, was a much better instrument. Directions we would pursue: architectures that estimate bias, gravity and time offset explicitly and with an uncertainty; self-supervised consistency between

the two IMUs of the same flight; and, above all, training data whose synchronisation has been verified.

A. Feedback to the Organizers

The rules, the metric and the starter kit were clear, and shipping the exact scorer was the single most useful thing in the kit: our local val floor (0.013) and all-zero score (1.015) matched the published references, so we trusted val throughout. What cost the most time was discovering the drone data properties of Sec. IV-D ourselves; a note on per-source conventions, IMU types and synchronisation quality (or a fix of the drone-B gyroscope sign and the racing-drone offsets) would have let us spend that time on the model. The metric behaved as documented, but per-trajectory averaging makes a 13-window drone clip worth as much as a 35-minute drive, and that is what makes the public leaderboard noisy; a per-window average within a platform, or a larger public split, would help. We would also suggest either merging or declaring the second-IMU duplicates of the racing flights. Finally, because the sources are recognisable from the signal, a platform-blind test set measures source recognition as much as generalisation; a genuinely held-out platform, or a causal evaluation track, would measure the latter more directly.

VI. CONCLUSION

A context network over whole trajectories, trained with a sampler that mirrors the metric and with physically exact augmentation, gives a single unified inertial velocity estimator with one set of weights and no post-processing. It scores 0.219 on the full test set and 0.286 on the harder public split, up from our own starting point of 0.553. What remains is dominated by one drone source whose ground truth disagrees with itself; closing that gap needs better data, or models that estimate synchronisation and mounting explicitly.

ACKNOWLEDGMENT

We thank the CMU AirLab team for the dataset, the exact scorer and the scoring service, and Kaggle and Google Colab for the compute. The code and analysis were developed with the help of the AI assistant Claude (Anthropic), under the authors' direction.

APPENDIX

A. Per-Sequence Results (Full Table)

B. Reproducibility and Submission Artifacts

Team and submissions. Kaggle team **Hack2Publish**. Ranked submission **56402554**

(`submission_final_kaggle_w_uni.csv`, 2026-09-20 17:34 UTC, md5

`fd12d4f25d7b52d44aaa4e4a14534378`, public 0.28609, private 0.16141, full test 0.21903). Second selected entry **56332266**

(`submission_final_v4_160.csv`, 2026-09-18 08:17 UTC, md5 `97a6e86dba7dfc19929e050bef5857b0`, public 0.28696, private 0.16625, full test 0.22189), the width-128 / 2-layer variant trained on the laptop with length-proportional sampling.

Code. GitHub repository <https://github.com/hamidhosen42/TartanIMU-Challenge-Multi-Platform-Inertial-Odometry>, training commit `a9a5384`; the Kaggle kernel script that trained the ranked checkpoint is in `release/kaggle_kernel/`.

Checkpoints. Hugging Face repository

<https://huggingface.co/mdhamidhosen/tartanimu-unified-hosen42>. `model.pt` (ranked entry; md5 `737c12af68ea4ad6aa419a72e92bad4d`, SHA-256 `69764745...`) and `model_secondary_final_v4_160.pt` (second entry; md5 `9ea78ce8819b5212d9d7983875f8ad42`). The exact `argparse` configuration is stored inside each checkpoint and in `models/configs/`. Full checksums are in SHA256SUMS.

Environment. Python 3.12, PyTorch 2.14.0, NumPy 2.5.2, pandas 3.0.5, as pinned in `requirements.txt`; inference needs no other package and no internet access.

Hardware and time. Ranked checkpoint: Kaggle Tesla T4, 3.6h. Inference: the whole test set in 26 s on a T4, 30 min on a 10-core CPU.

Seeds and spread. Seed 0 everywhere; every reported score is a single run. The run-to-run spread on the public leaderboard is large: the same recipe trained on two different GPUs scored 0.2870 and 0.3052, and the ranked recipe scored 0.2861 on the Kaggle GPU and 0.2876 on the laptop (0.1614 and 0.1560 on the private leaderboard). Inference itself is deterministic up to device arithmetic; re-running `infer.py` on another device reproduces the submitted CSV to within 0.005 m/s per window (TF32 disabled).

Compliance declaration.

- 1) Yes. All ranked predictions come from a single model with one shared set of weights.
- 2) No platform classification, routing or specialisation is used at inference. A platform classification head exists in the training graph only; at inference its output is not computed and nothing depends on it.
- 3) No model ensembling and no test-time augmentation. The only weight averaging is inside one training run: an EMA of the weights during training and the plain average of the EMA weights over the last 20 epochs.
- 4) The test data was read only by the inference script. Architecture and hyper-parameters were chosen on the released `val` split with the organisers' scorer, and the final model is a fixed-schedule refit on `train+val` with no checkpoint selection. Among the finished refits we chose the two Kaggle entries by `val`-validated recipe first and public score second; nothing in training or post-processing was tuned against the leaderboard, and no test information was used in any other way.
- 5) The development-time system and the submitted system are identical. The ranked and the second entry differ only in trunk width, transformer depth and the sampling weights stated above.

TABLE V

PER-SEQUENCE CHALLENGE RESULTS OVER THE 89 TEST SEQUENCES (OFFICIAL PER-SEQUENCE SCORING SERVICE; SUBMISSION MD5 FD12D4F25D7B52D44AAA4E4A14534378). **RTE** (RELATIVE TRAJECTORY ERROR, M) FOLLOWS STURM *et al.* [2]: OVER SLIDING WINDOWS OF $\Delta t = 5$ s, THE RMSE OF THE DIFFERENCE BETWEEN PREDICTED AND GROUND-TRUTH RELATIVE DISPLACEMENT,
$$\text{RTE} = \sqrt{\frac{1}{n} \sum_i \|(\hat{\mathbf{p}}_{i+\Delta t} - \hat{\mathbf{p}}_i) - (\mathbf{p}_{i+\Delta t} - \mathbf{p}_i)\|^2}.$$
 SEQUENCE IDS FOLLOW THE SCORING SERVICE'S ORDERING.

Seq.	ATE ₂₀	AVE	RTE	Seq.	ATE ₂₀	AVE	RTE	Seq.	ATE ₂₀	AVE	RTE
1	0.830	0.337	1.873	31	0.703	0.298	1.605	61	0.906	0.360	1.158
2	0.498	0.351	1.181	32	0.319	0.084	0.246	62	0.073	0.020	0.061
3	0.535	0.205	0.765	33	0.210	0.051	0.207	63	0.455	0.070	0.327
4	0.498	0.308	0.903	34	1.581	0.366	1.800	64	0.404	0.059	0.268
5	0.905	0.405	1.835	35	1.056	0.413	1.659	65	0.106	0.038	0.130
6	0.674	0.089	0.763	36	1.016	0.325	1.878	66	2.153	0.226	1.577
7	1.116	0.344	1.508	37	2.555	2.108	11.718	67	0.113	0.032	0.143
8	0.468	0.039	0.208	38	2.152	1.807	8.917	68	0.526	0.048	0.222
9	0.640	0.278	0.972	39	0.644	0.256	0.977	69	0.686	0.084	0.440
10	0.771	0.086	0.693	40	0.784	0.318	1.853	70	0.503	0.073	0.330
11	1.024	0.658	2.017	41	1.081	0.327	2.140	71	0.196	0.060	0.192
12	0.307	0.154	0.672	42	0.493	0.088	0.483	72	0.851	0.452	1.437
13	0.300	0.044	0.262	43	0.920	0.286	1.115	73	0.807	0.131	0.767
14	0.564	0.383	1.228	44	0.212	0.051	0.315	74	0.595	0.310	1.006
15	1.021	0.073	0.409	45	0.083	0.053	0.126	75	0.548	0.393	1.216
16	0.127	0.034	0.224	46	0.258	0.059	0.241	76	0.399	0.097	0.423
17	1.136	0.559	2.669	47	0.840	0.387	1.063	77	1.166	0.567	2.337
18	1.360	0.565	2.554	48	0.211	0.015	0.101	78	0.197	0.029	0.098
19	0.521	0.294	1.308	49	0.798	0.255	1.343	79	0.817	0.261	1.419
20	0.140	0.053	0.370	50	0.669	0.210	1.117	80	0.448	0.174	0.910
21	0.706	0.209	1.133	51	0.080	0.037	0.097	81	0.645	0.116	0.474
22	0.573	0.391	1.197	52	0.258	0.070	0.359	82	2.896	0.389	2.533
23	1.027	0.241	1.984	53	0.394	0.279	0.513	83	0.449	0.074	0.372
24	0.392	0.043	0.193	54	0.222	0.070	0.260	84	0.477	0.057	0.321
25	0.105	0.029	0.109	55	0.269	0.144	0.665	85	0.074	0.032	0.097
26	0.596	0.086	0.353	56	3.406	2.896	15.349	86	0.305	0.027	0.148
27	0.476	0.060	0.290	57	0.922	0.356	2.145	87	0.414	0.173	0.657
28	0.566	0.256	1.271	58	0.219	0.050	0.250	88	1.028	0.294	2.012
29	3.661	2.886	16.039	59	0.361	0.074	0.385	89	0.148	0.028	0.091
30	1.136	0.548	2.608	60	0.156	0.066	0.214				

C. Final Predictions

The ranked CSV (30,644 rows, header window_id, vx, vy, vz, md5 above) is the file attached to the Submission Form. It is regenerated from the Hugging Face repository by

```
python infer.py --traj_dir test
--windows index/test_windows.csv --out
submission.csv
```

(bit-exact on the training device). Nothing is applied after the network output except the overlap averaging of Sec. II-E. There are no missing or degenerate windows; every test window receives a prediction.

D. Model and Inference Code

Weights, the inference entry point, the configuration and the pinned environment (Hugging Face):

<https://huggingface.co/mdhamidhosen/tartanimu-unified-hosen42>

Training code, analysis scripts and the sources of this report (GitHub):

<https://github.com/hamidhosen42/TartanIMU-Challenge-Multi-Platform-Inertial-Odometry>

REFERENCES

- [1] S. Zhao, S. Zhou, R. Blanchard, Y. Qiu, W. Wang, and S. Scherer, "Tartan IMU: A Light Foundation Model for Inertial Positioning in Robotics," in *Proc. IEEE/CVF CVPR*, 2025, pp. 22520–22529.
- [2] J. Sturm, N. Engelhard, F. Endres, W. Burgard, and D. Cremers, "A Benchmark for the Evaluation of RGB-D SLAM Systems," in *Proc. IEEE/RSJ IROS*, 2012, pp. 573–580.
- [3] S. Herath, H. Yan, and Y. Furukawa, "RoNIN: Robust Neural Inertial Navigation in the Wild," in *Proc. IEEE ICRA*, 2020.
- [4] W. Liu, D. Caruso, E. Ilg, J. Dong, A. Angelova, and S. Belongie, "TLIO: Tight Learned Inertial Odometry," *IEEE Robot. Autom. Lett.*, vol. 5, no. 4, 2020.